

Technical Report

Prestige Eventos & Recepciones: A Transactional Web System Designed with Object-Oriented Programming

Authors:

Luisa Daniela Correa Torres – 20242020048

Juan Pablo Rodríguez – 20251020072

Institution:

Universidad Distrital Francisco José de Caldas

Date: October 2025

Contents

Abstract	3
1 Introduction	4
2 Literature Review	5
3 Background	6
4 Objectives	7
4.1 General Objective	7
4.2 Specific Objectives	7
5 Scope	7
6 Assumptions	8
7 Limitations	8
8 Methodology	9
8.1 Object-Oriented Design	9
8.2 UML and CRC Modeling	9
8.3 CRC Cards	10
9 Results	11
10 Discussion	11
11 Conclusion	11
12 References	12
13 Appendices	12
14 Glossary	12

List of Figures

1	General architecture of the system showing class interactions between Client, Reservation, Service, Payment, and Calendar.	6
2	UML Class Diagram of the Prestige Eventos & Recepciones system showing relationships between Client, Reservation, Service, Payment, and Calendar. .	9
3	CRC 1: Class Venue, Class User	10
4	CRC 2: Class Calendar, Class Payment	10
5	CRC 3: Class Service, Class Reservation	10

Abstract

This technical report presents the design and object-oriented structure of Prestige Eventos Recepciones, a platform developed in Java to simplify the event planning process. The project applies Object-Oriented Programming (OOP) principles such as encapsulation, inheritance, abstraction, and polymorphism to create a modular, reusable, and secure system.

The system integrates multiple event services—venue selection, catering, and decoration—into one unified environment. Through the use of UML diagrams, CRC cards, and Java class implementations, the report demonstrates how OOP supports clear architecture, scalability, and maintainability.

1. Introduction

Planning large events involves coordinating various providers, services, and logistics—often leading to inefficiency and user frustration. *Prestige Eventos & Recepciones* addresses this problem by creating a web-based platform where clients can register, select predefined service packages, view availability through an interactive calendar, and make secure online payments.

This report focuses on the object-oriented design of the system, emphasizing how OOP principles were applied to structure the platform’s classes, interactions, and responsibilities. Each component of the system corresponds to a real-world concept—users, reservations, services, payments—and communicates through method calls and collaborations.

The project demonstrates how applying OOP to web application design enables modularity, ease of modification, and data security, setting a foundation for professional, maintainable software development.

2. Literature Review

Object-Oriented Programming (OOP) is one of the most influential paradigms in modern programming. It structures code around objects, which encapsulate both data and behavior. According to Gálvez and Guerrero (2018), UML 2.5 provides a standard language for describing object-oriented systems through diagrams that express relationships and responsibilities.

OOP principles have been proven to improve:

- **Reusability:** allowing components to be extended for new use cases.
- **Encapsulation:** protecting data through controlled access.
- **Abstraction:** reducing system complexity by focusing on essentials.
- **Maintenance:** isolating changes within independent modules.

The use of UML and CRC cards in OOP design facilitates early visualization of class responsibilities and interactions before any code is written. This ensures that conceptual design aligns with implementation.

Java's architecture enforces OOP patterns through its syntax, access control, and modularity, making it ideal for developing transactional systems that prioritize flexibility and security.

3. Background

The project was conceived to demonstrate the practical application of Object-Oriented Programming in developing a transactional web system. The goal was to model the real-world process of event organization—registration, service selection, reservation, and payment—through interacting software objects.

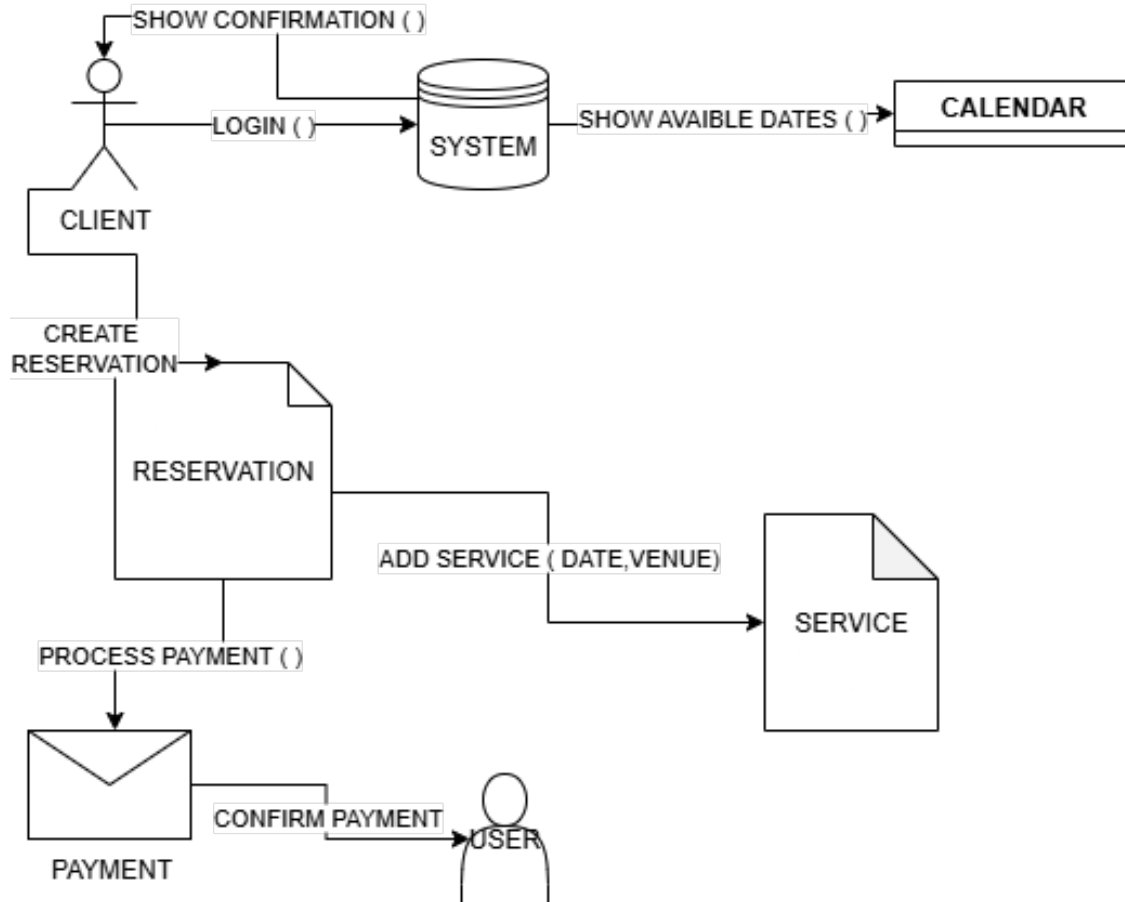


Figure 1: General architecture of the system showing class interactions between Client, Reservation, Service, Payment, and Calendar.

Each element of event management is represented by a class: **Client**, **Reservation**, **Service**, **Payment**, and **Calendar**. These objects interact to create a functional, modular structure.

By employing CRC cards and UML class diagrams, the project achieves a design that is both conceptually clear and technically feasible, providing a framework for future implementation and expansion.

4. Objectives

4.1. General Objective

To design and model a transactional application for event booking using Object-Oriented Programming principles.

4.2. Specific Objectives

- Apply OOP principles—encapsulation, inheritance, abstraction, and polymorphism—to define system architecture.
- Develop UML and CRC documentation to represent class responsibilities and collaborations.
- Implement Java class prototypes to simulate interaction between main components.
- Demonstrate how OOP facilitates modularity, reusability, and code maintainability.

5. Scope

The report covers the conceptual and technical design of the system using OOP principles. It includes:

- Requirement-based class design.
- UML and CRC documentation.
- Java class prototypes for system components.
- Interaction examples demonstrating OOP behavior.

It excludes:

- Real payment processing and external API integration.
- Full database integration.

6. Assumptions

- The system will be implemented in Java using object-oriented design.
- Data protection will rely on encapsulation and controlled access.
- Future extensions will include subclasses for new services and payment types.
- UML and CRC models will remain consistent with implementation.

7. Limitations

- There is no database integration.
- Integration with payment gateways and real data storage is pending.

8. Methodology

The system was designed using Object-Oriented Programming as the guiding paradigm. The process included identifying entities, defining relationships, and simulating behaviors through class interactions.

8.1. Object-Oriented Design

Objects represent real entities: **Client**, **Reservation**, **Service**, **Payment**, and **Calendar**. Each class encapsulates its own attributes and behaviors.

8.2. UML and CRC Modeling

UML Class Diagram: The UML diagram illustrates class relationships, showing composition between Reservation and Service, and associations between Client and Payment. This helps visualize data flow and responsibility distribution, core principles of OOP design.

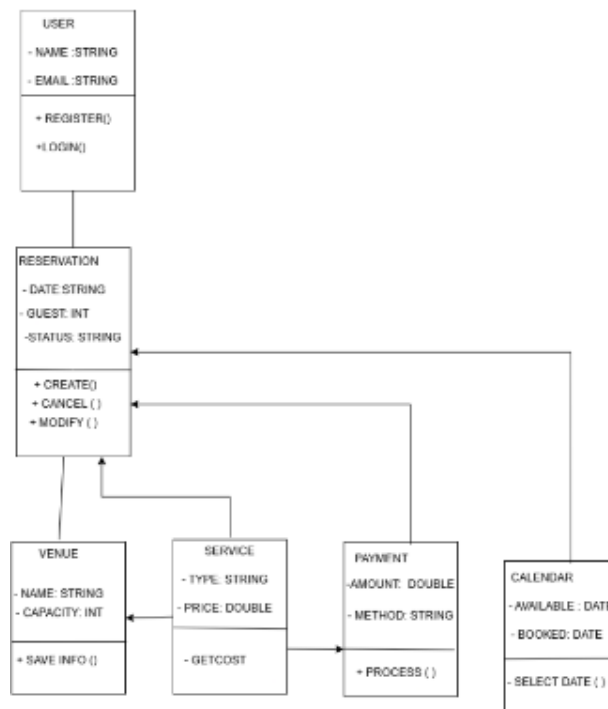


Figure 2: UML Class Diagram of the Prestige Eventos & Recepciones system showing relationships between Client, Reservation, Service, Payment, and Calendar.

8.3. CRC Cards

Each CRC card defines what a class does and which other classes it collaborates with.

Class: Venue	Class: User
Responsibilities: • Save place info • Check dates • Verify capacity • Manage calendar	Responsibilities: • Create accounts • Save user data • Manage preferences
Collaborators: • Reservation • Calendar	Collaborators: • Reservation • Payment

Figure 3: CRC 1: Class Venue, Class User

Class: Calendar	Class: Payment
Responsibilities: • Show dates • Block dates • Date selection	Responsibilities: • Process payments • Payment methods • Generate receipts • Handle refunds
Collaborators: • Venue • Reservation	Collaborators: • Reservation • User

Figure 4: CRC 2: Class Calendar, Class Payment

Class: Service	Class: Reservation
Responsibilities: • Manage services • Save prices • Check availability • Calculate cost	Responsibilities: • Create reservations • Calculate total • Change status • Manage dates
Collaborators: • Reservation	Collaborators: • User • Venue • Service • Payment

Figure 5: CRC 3: Class Service, Class Reservation

9. Results

The main results achieved are:

- A fully defined object-oriented class structure.
- UML and CRC documentation that explains all class interactions.
- A working simulation of core processes (registration, reservation, payment).
- Encapsulated Java classes ensuring secure data handling.

The results confirm that OOP provides a clear and scalable architecture for transactional systems.

10. Discussion

Applying Object-Oriented Programming enabled a logical and reusable system structure. Each class mirrors a real-world element of event management.

The design ensures:

- Independence between components.
- Easy maintenance through modular classes.
- Potential for polymorphism in extended services.

This structure demonstrates that OOP is not only a coding technique but a design strategy that improves scalability and security in software projects.

11. Conclusion

The object-oriented design of *Prestige Eventos & Recepciones* successfully transforms a complex real-world problem into a structured digital solution.

Key conclusions:

- OOP allows natural modeling of entities and relationships.
- Encapsulation secures user and transaction data.

- Inheritance and polymorphism support scalability.
- UML and CRC tools ensure clarity in design and communication.

The project establishes a strong foundation for the next implementation stage, where databases, payment gateways, and interfaces can be incorporated seamlessly.

12. References

- Gálvez, J., & Guerrero, J. (2018). *UML 2.5: Object-Oriented Software Modeling*. RA-MA Editorial.
- Oracle. (2024). *The Java Tutorials – Object-Oriented Programming Concepts*. Retrieved from <https://docs.oracle.com>
- Beck, K., & Cunningham, W. (1989). *Using CRC Cards for Object-Oriented Design*.

13. Appendices

Appendix A: CRC Cards of main classes.

Appendix B: UML Class Diagram.

Appendix C: Java Source Code snippets.

Appendix D: User Story tables.

14. Glossary

Class	Blueprint defining attributes and methods for objects.
Object	Instance of a class representing a real-world entity.
Encapsulation	Protecting internal object data using private access.
Abstraction	Modeling only essential features of an entity.
Inheritance	Mechanism for code reuse between related classes.
Polymorphism	Ability of different classes to respond uniquely to the same method.
UML	Unified Modeling Language for visualizing object-oriented systems.

CRC Card

Tool for defining class responsibilities and collaborators.